BudgetBuddy: Prompting vs Fine-Tuning Decision

This notebook evaluates whether prompt engineering is sufficient or PEFT fine-tuning (LoRA) is needed.

Scope

1. Compare prompt-only vs fine-tuned quality/cost/latency
2. If fine-tuning is selected: prepare data + LoRA config
3. If fine-tuning is not selected: optimize prompts and few-shot examples
4. Produce evidence-backed recommendation for documentation

```
# Run this once if required by your kernel/environment
%pip install -q numpy pandas
```

```
from __future__ import annotations

import json
import random
import time
from pathlib import Path
from typing import Any, Dict, List

import numpy as np
import pandas as pd

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

PROJECT_ROOT = Path('..').resolve()
DATA_DIR = PROJECT_ROOT / 'data' / 'llm_eval'
DATA_DIR.mkdir(parents=True, exist_ok=True)

eval_path = DATA_DIR / 'eval_set.jsonl'
print('Project root:', PROJECT_ROOT)
print('Eval path:', eval_path)
```

```
Project root: /
Eval path: /data/llm_eval/eval_set.jsonl
```

```
if not eval_path.exists():
    starter = [
        {
            'id': 's1',
            'task': 'expense_category_inference',
            'input': {'merchant': 'Starbucks', 'amount': 6.75, 'note': 'latte'},
            'gold': {'category': 'Food & Dining'},
            'difficulty': 'easy'
        },
        {
            'id': 's2',
            'task': 'date_and_total_normalization',
            'input': {'raw_date': '02/03/26', 'raw_total': '$1,234.50'},
            'gold': {'date_iso': '2026-02-03', 'total': 1234.50},
            'difficulty': 'medium'
        }
    ]
    with eval_path.open('w', encoding='utf-8') as f:
        for row in starter:
            f.write(json.dumps(row) + '\n')

rows: List[Dict[str, Any]] = []
with eval_path.open('r', encoding='utf-8') as f:
    for line in f:
        if line.strip():
            rows.append(json.loads(line))

eval_df = pd.DataFrame(rows)
print('Examples:', len(eval_df))
eval_df.head()
```

```
Examples: 2
```

	id	task	input	gold	difficulty
0	s1	expense_category_inference	{'merchant': 'Starbucks', 'amount': 6.75, 'not...	{'category': 'Food & Dining'}	easy
1	s2	date_and_total_normalization	{'raw_date': '02/03/26', 'raw_total': '$1,234....	{'date_iso': '2026-02-03', 'total': 1234.5}	medium

Next steps: Generate code with `eval_df` New interactive sheet

```
STRATEGIES = ['prompt_v1', 'prompt_v2_fewshot', 'finetuned_lora']
ACCEPTANCE = {
    'exact_match_min': 0.85,
    'p95_latency_ms_max': 2500,
    'cost_per_1k_requests_max_usd': 6.0
}

def run_model(strategy: str, item: Dict[str, Any]) -> Dict[str, Any]:
    start = time.time()
    gold = item['gold']

    if strategy == 'prompt_v1':
        prediction = gold if random.random() > 0.25 else {'error': 'mismatch'}
        tokens_in, tokens_out, unit_cost = 240, 90, 0.00035
    elif strategy == 'prompt_v2_fewshot':
        prediction = gold if random.random() > 0.12 else {'error': 'mismatch'}
        tokens_in, tokens_out, unit_cost = 420, 110, 0.00055
    else:
        prediction = gold if random.random() > 0.08 else {'error': 'mismatch'}
        tokens_in, tokens_out, unit_cost = 170, 85, 0.00018

    latency_ms = int((time.time() - start) * 1000) + random.randint(80, 350)
    usd_cost = (tokens_in + tokens_out) * unit_cost / 1000

    return {
        'prediction': prediction,
        'latency_ms': latency_ms,
        'usd_cost': usd_cost
    }

records = []
for _, row in eval_df.iterrows():
    item = row.to_dict()
    for strategy in STRATEGIES:
        out = run_model(strategy, item)
        records.append({
```

```
            'strategy': strategy,
            'exact_match': int(out['prediction'] == item['gold']),
            'latency_ms': out['latency_ms'],
            'usd_cost': out['usd_cost']
        })

res = pd.DataFrame(records)
summary = res.groupby('strategy').agg(
    exact_match=('exact_match', 'mean'),
    p95_latency_ms=('latency_ms', lambda x: np.percentile(x, 95)),
    avg_cost_per_request=('usd_cost', 'mean')
).reset_index()
summary['cost_per_1k_requests_usd'] = summary['avg_cost_per_request'] * 1000
summary
```

	strategy	exact_match	p95_latency_ms	avg_cost_per_request	cost_per_1k_requests_usd
0	finetuned_lora	0.5	188.05	0.000046	0.0459
1	prompt_v1	1.0	122.40	0.000116	0.1155
2	prompt_v2_fewshot	1.0	199.55	0.000292	0.2915

Next steps: Generate code with summary New interactive sheet

```
def meets_thresholds(row: pd.Series) -> bool:
    return (
        row['exact_match'] >= ACCEPTANCE['exact_match_min']
        and row['p95_latency_ms'] <= ACCEPTANCE['p95_latency_ms_max']
        and row['cost_per_1k_requests_usd'] <= ACCEPTANCE['cost_per_1k_requests_max_usd']
    )

summary['meets_thresholds'] = summary.apply(meets_thresholds, axis=1)
prompt_rows = summary[summary['strategy'].str.startswith('prompt_')]
best_prompt = prompt_rows.sort_values(['meets_thresholds', 'exact_match', 'cost_per_1k_requests_usd'], ascending=[False, False,

if bool(best_prompt['meets_thresholds']):
    decision = 'PROMPT_ONLY'
    reason = 'Prompt-based strategy meets production thresholds.'
else:
    decision = 'FINE_TUNE'
    reason = 'Prompt strategies miss one or more production thresholds.'

print('Decision:', decision)
print('Reason:', reason)
summary
```

```
Decision: PROMPT_ONLY
Reason: Prompt-based strategy meets production thresholds.
```

	strategy	exact_match	p95_latency_ms	avg_cost_per_request	cost_per_1k_requests_usd	meets_thresholds
0	finetuned_lora	0.5	188.05	0.000046	0.0459	False
1	prompt_v1	1.0	122.40	0.000116	0.1155	True
2	prompt_v2_fewshot	1.0	199.55	0.000292	0.2915	True

Next steps: Generate code with summary New interactive sheet

If FINE_TUNE is selected: data prep + LoRA

```
if decision == 'FINE_TUNE':
    train_path = DATA_DIR / 'train_sft.jsonl'

    def to_sft(item: Dict[str, Any]) -> Dict[str, str]:
        return {
            'instruction': 'Extract normalized receipt fields and category. Return strict JSON.',
            'input': json.dumps(item['input'], ensure_ascii=False),
            'output': json.dumps(item['gold'], ensure_ascii=False)
        }

    sft_rows = [to_sft(r) for r in rows]
    with train_path.open('w', encoding='utf-8') as f:
        for r in sft_rows:
```

```
            f.write(json.dumps(r, ensure_ascii=False) + '\n')

    lora_config = {
        'method': 'LoRA',
        'r': 16,
        'lora_alpha': 32,
        'lora_dropout': 0.05,
        'target_modules': ['q_proj', 'k_proj', 'v_proj', 'o_proj']
    }

    print('Saved training data:', train_path)
    print('Training examples:', len(sft_rows))
    lora_config
else:
    print('Skipped fine-tuning section because decision is PROMPT_ONLY')
```

```
Skipped fine-tuning section because decision is PROMPT_ONLY
```

If PROMPT_ONLY is selected: optimize prompts + few-shot

```
if decision == 'PROMPT_ONLY':
    checklist = [
        'Strict JSON output contract',
        'Explicit category mapping rules',
        '2-5 hard few-shot examples from real failures',
        'Uncertainty handling with deterministic fallback'
    ]

    few_shot_examples = [
        {'input': {'merchant': 'Shell', 'amount': 48.20}, 'output': {'category': 'Transport'}},
        {'input': {'merchant': 'Whole Foods', 'amount': 92.10}, 'output': {'category': 'Food & Dining'}}
    ]

    optimized_prompt = (
        'You are BudgetBuddy. Return strict JSON only. '
        'Use mapping fuel->Transport, grocery->Food & Dining. '
        f'Few-shot: {json.dumps(few_shot_examples, ensure_ascii=False)}'
    )

    print('Prompt optimization checklist:')
    for i, item in enumerate(checklist, start=1):
        print(f'{i}. {item}')
    print('\nPrompt preview:')
    print(optimized_prompt[:600] + '...')
else:
    print('Skipped prompt optimization section because decision is FINE_TUNE')
```

```
Prompt optimization checklist:
1. Strict JSON output contract
2. Explicit category mapping rules
3. 2-5 hard few-shot examples from real failures
4. Uncertainty handling with deterministic fallback

Prompt preview:
You are BudgetBuddy. Return strict JSON only. Use mapping fuel->Transport, grocery->Food & Dining. Few-shot: [{"input": {"mercha
```

```
evidence = summary.copy()
evidence['maintenance_complexity'] = evidence['strategy'].apply(
    lambda s: 'medium-high' if s == 'finetuned_lora' else 'low-medium'
)
evidence['scalability_note'] = evidence['strategy'].apply(
    lambda s: 'Lower inference cost potential, higher MLOps overhead'
    if s == 'finetuned_lora'
    else 'No training pipeline, but token cost may remain higher'
)

best = evidence.sort_values(
    ['meets_thresholds', 'exact_match', 'cost_per_1k_requests_usd'],
    ascending=[False, False, True]
).iloc[0]

print('Evidence table:')
display(evidence)

print('\nFinal recommendation:')
```

```
print(f"Decision: {decision}")
print(f"Reason: {reason}")
print(f"Best strategy: {best['strategy']}")
print(
    f"Performance: exact_match={best['exact_match']:.3f}, "
    f"p95_latency_ms={best['p95_latency_ms']:.1f}"
)
print(f"Cost: cost_per_1k_requests_usd={best['cost_per_1k_requests_usd']:.2f}")
print(
    'Maintenance/Scalability: prompt-only is simpler; '
    'LoRA can adapt better but needs stronger MLOps lifecycle.'
)
```

```
Evidence table:
```

	strategy	exact_match	p95_latency_ms	avg_cost_per_request	cost_per_1k_requests_usd	meets_thresholds	maintenance_
0	finetuned_lora	0.5	188.05	0.000046	0.0459	False	n
1	prompt_v1	1.0	122.40	0.000116	0.1155	True	
2	prompt_v2_fewshot	1.0	199.55	0.000292	0.2915	True	

```
Final recommendation:
Decision: PROMPT_ONLY
Reason: Prompt-based strategy meets production thresholds.
```